CSA1741-AI
EXPERIMENTS 1-10

Exp-1

Write a python program to implement Simple Calculator program

Aim:

To write a python program to implement simple calculator.

Algorithm:

Step 1: Start by asking the user to input two numbers and the desired

Step 2: If the operation is addition (+), add the two numbers and display the result.

Step 3:If the operation is subtraction (-), subtract the second number from the first
number and display the result. If the.

Step 4: If the operation is division (/), divide the first number by the second number and
display the result.

Step 5: if the user does not want to perform another calculation, end the program.

Program:

```
def add(x, y):
return x + y
def subtract(x, y):
return x - y
def multiply(x, y):
return x * y
def divide(x, y):
return x / y
print("Select operation.")
print("1.Add")
print("2.Subtract")
print("3.Multiply")
print("4.Divide")
choice = input("Enter choice(1/2/3/4): ")
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
if choice == '1':
print(num1,"+",num2,"=", add(num1,num2))
elif choice == '2':
print(num1,"-",num2,"=", subtract(num1,num2))
elif choice == '3':
print(num1,"*",num2,"=", multiply(num1,num2))
```

```
elif choice == '4':
print(num1,"/",num2,"=", divide(num1,num2))
else:
print("Invalid input")
```

Output:

```
Select operation.
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 1
Enter first number: 10
Enter second number: 5
10.0 + 5.0 = 15.0
```

Result:

Hence, the simple python program for multiplication tabl is done

Exp-2 Write a python program to Add Two Matrices

Aim:

To write a python program to add two matrices.

Algorithm:

Step1 : Start by initializing two matrices, matrix A and matrix B, of the same
size (m x n).

Step 2: Initialize an empty matrix, matrix C, also of size m x n.

Step 3: Use a nested loop to iterate over each element in matrix A and matrix B:

Step 4:Add the corresponding elements of matrix A and matrix B and store the
result in the corresponding element of matrix C.

Step 5:Display the resulting matrix C.

Program:

```
matrix1 = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]
matrix2 = [[10, 11, 12],
[13, 14, 15],
[16, 17, 18]]
result = [[0, 0, 0],
[0, 0, 0],
[0, 0, 0]]
for i in range(len(matrix1)):
```

```
for j in range(len(matrix1[0])):
# Add corresponding elements of matrices
result[i][j] = matrix1[i][j] + matrix2[i][j]
for row in result:
print(row)
```

Output:

```
[11, 13, 15]
[17, 19, 21]
[23, 25, 27]
```

Result:
Hence, the simple python program for matrix table is done.

Exp-3 Write a python program to Transpose a Matrix.

Aim: To write a python program to transpose a matrix.

Algorithm:
Step 1:Start by initializing the original matrix, matrix A, of size m x n.
Step 2:Initialize an empty matrix, matrix B, of size n x m.
Step 3:Use a nested loop to iterate over each element in matrix A:
Step 4: Store the value of the current element in matrix A in the corresponding
element in matrix B, but with the row and column indices swapped.
Step 5: Display the resulting matrix B.

Program:

```
matrix = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]
rows = len(matrix)
cols = len(matrix[0])
transpose_matrix = [[0 for x in range(rows)] for y in range(cols)]
for i in range(rows):
for j in range(cols):
transpose_matrix[j][i] = matrix[i][j]
print("Original Matrix:")
for row in matrix:
print(row)
print("Transposed Matrix:")
for row in transpose_matrix:
print(row)
```

Output: Original Matrix:

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
Transposed Matrix:
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
Result: Hence, the simple python program for transpose a matrix is done.

Exp -4 Write a python program to sort the sentence in alphabetical order

Aim:
To write a python program to sort the sentence in alphabetical order.
Algorithm:
Step 1: Start by taking input of the sentence.
Step 2: Split the sentence into individual words and store them in a list.
Step 3: Use the sort() method to sort the list of words in alphabetical order.
Step 4: Join the sorted list of words back into a sentence.
Step 5: Display the resulting sorted sentence.
Program:

```
sentence = input("Enter a sentence: ")
words = sorted(sentence.split())
sorted_sentence = " ".join(words)
print("Sorted sentence:", sorted_sentence)
```

OutPut:
Enter a sentence: banana apple mango orange
Sorted sentence: apple banana mango orange

Result: Hence, the simple python program for sort the sentence in alphabetical order is done

Exp-5 Write a python program to implement List operations (Nested List, Length, Concatenation,Membership, Iteration, Indexing and Slicing)
Aim: To write a python program to implement the given list operations.
Algorithm:
Step 1: Start by initializing a list.
Step 2: Use the len() function to find the length of the list.
Step 3: Use concatenation to join two lists.
Step 4: Use the in keyword to check if an element is in the list.
Step 5: Use a for loop to iterate over the list.

Step 6: Use indexing to access a specific element in the list.
Step 7: Use slicing to access a range of elements in the list.

Program:

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print("Nested List:", nested_list)
my_list = [1, 2, 3, 4, 5]
print("Length of List:", len(my_list))
list1 = [1, 2, 3]
list2 = [4, 5, 6]
concatenated_list = list1 + list2
print("Concatenated List:", concatenated_list)
my_list = [1, 2, 3, 4, 5]
if 3 in my_list:
print("3 is in the list")
else:
print("3 is not in the list")
my_list = [1, 2, 3, 4, 5]
for item in my_list:
print(item)
my_list = [1, 2, 3, 4, 5]
print("First element:", my_list[0])
print("Last element:", my_list[-1])
print("Sliced List:", my_list[1:4])
```

Output:

```
Nested List: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Length of List: 5
Concatenated List: [1, 2, 3, 4, 5, 6]
3 is in the list
1
2
3
4
5
First element: 1
Last element: 5
Sliced List: [2, 3, 4]
```

Result: Hence, the simple python program for implement the operations is done

Exp-6 Write a python program to implement Listmethods (Add, Append, Extend & Delete).

Aim: To write a python program to implement the given list methods.

Algorithm:

Step 1: Start by initializing a list.

Step2: Use the append() method to add an element to the end of the list.

Step3: Use the extend() method to add multiple elements to the end of the list.

Step4: Use the insert() method to add an element at a specific index in the list.

Step5: Use the remove() method to remove a specific element from the list.

Step6: Use the pop() method to remove and return the last element of the list, or a
specific element by index.

Step 7: Use the del keyword to delete an element or slice from the list.

Program:

```
my_list = []
my_list.add(1)
print("List after adding an item using add method:", my_list)
my_list.append(2)
print("List after appending an item using append method:", my_list)
my_list.extend([3, 4, 5])
print("List after extending using extend method:", my_list)
my_list.remove(3)
print("List after removing an item using remove method:", my_list)
```

Output:

```
List after adding an item using add method: [1]
List after appending an item using append method: [1, 2]
List after extending using extend method: [1, 2, 3, 4, 5]
List after removing an item using remove method: [1, 2, 4, 5]
```

Result:

Hence, the simple python program for implement the list of elements done

Exp-7 Write a Python Program to Illustrate Different Set Operations

Aim: To write a python program to illustrate different set operations.

Algorithm:

Step1: Create two sets setA and setB containing some elements.

Step2: Perform the intersection operation on the sets by using the intersection()
method and store the result in a new set intersection_set.

Step3: Perform the union operation on the sets by using the union() method and store
the result in a new set union_set.

Step4: Perform the difference operation on the sets by using the difference() method
and store the result in a new set difference_set.
Step5: Check if setA is a subset of setB by using the issubset() method and print the
result.

Program:

```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}
print("Set 1:", set1)
print("Set 2:", set2)
print("Intersection of set1 and set2:", intersection_set)
union_set = set1.union(set2)
print("Union of set1 and set2:", union_set)
difference_set = set1.difference(set2)
print("Difference between set1 and set2:", difference_set)
symmetric_difference_set = set1.symmetric_difference(set2)
print("Symmetric difference between set1 and set2:", symmetric_difference_set)
subset_check = set1.issubset(set2)
print("Set 1 is subset of set 2:", subset_check)
```

Output:

```
Set 1: {1, 2, 3, 4, 5}
Set 2: {4, 5, 6, 7, 8}
Intersection of set1 and set2: {4, 5}
Union of set1 and set2: {1, 2, 3, 4, 5, 6, 7, 8}
Difference between set1 and set2: {1, 2, 3}
Symmetric difference between set1 and set2: {1, 2, 3, 6, 7, 8}
Set 1 is subset of set 2: False
```

Result: Hence, the simple python program for different set of operations is done

Exp-8 Write a python program to generate Calendar for the given month and year

Aim:

To write a python program to generate calendar for the given month and year.

Algorithm:

Step1: Take the input month and year from the user.
Step2: Calculate the total number of days in the given month using a formula that
takes into account whether it's a leap year or not.

Step3: Determine the day of the week for the first day of the given month using
Zeller's congruence formula or any other method to calculate day of the week.
Step4: Print the header for the calendar.
Step5: Print the days of the week as column headers.
Step6: Print the resulting calendar.

Program:

```
import calendar
year = int(input("Enter the year: "))
month = int(input("Enter the month: "))
print(calendar.month(year, month))
```

Output:

```
Enter the year: 2026
Enter the month: 10
    October 2026
Mo Tu We Th Fr Sa Su
         1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

Result: Hence, the simple python program for calendar is done.

Exp-9 Write a Python Program to Remove punctuations from the given String

Aim: To write a python program to remove punctuations from the given string.

Program:

```
import string
input_string = input("Enter a string: ")
no_punctuations = input_string.translate(str.maketrans("", "", string.punctuation))
print("String with no punctuations:", no_punctuations)
```

Output:

```
Enter a string: Hello, World! How are you?
String with no punctuations: Hello World How are you
```

Result: Hence, the simple python program for punctuations is done.

Exp-10 Write the Python Program to solve 8-Puzzle Problem.

Aim: To write a python program to solve the 8-puzzle problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.
Step2: If the dequeued state is the goal state, return the solution path.
Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its
neighboring tiles.
Step5: For each successor state, calculate its priority based on the Manhattan distance
and add it to the frontier if it has not been explored before.

Program:

```
from queue import PriorityQueue
goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)
def heuristic(state):
in range(3):
for j in range(3):
tile = state[i * 3 + j]
if tile != 0:
x, y = (tile - 1) // 3, (tile - 1) % 3
distance += abs(x - i) + abs(y - j)
return distance
def solve(initial_state):
frontier = PriorityQueue()
frontier.put((heuristic(initial_state), initial_state))
explored = set()
while not frontier.empty():
_, state = frontier.get()
if state == goal_state:
return True
explored.add(state)
for successor in successors(state):
if successor not in explored:
priority = heuristic(successor) + len(explored)
frontier.put((priority, successor))
return False
def successors(state):
successors = []
i = state.index(0)
if i % 3 != 0:
# Slide tile to the left
new_state = list(state)
new_state[i], new_state[i - 1] = new_state[i - 1], new_state[i]
successors.append(tuple(new_state))
if i % 3 != 2:
# Slide tile to the right
new_state = list(state)
new_state[i], new_state[i + 1] = new_state[i + 1], new_state[i]
```

Output:

```
successors.append(tuple(new_state))
if i // 3 != 0:
# Slide tile up
new_state = list(state)
new_state[i], new_state[i - 3] = new_state[i - 3], new_state[i]
successors.append(tuple(new_state))
if i // 3 != 2:
# Slide tile down
new_state = list(state)
new_state[i], new_state[i + 3] = new_state[i + 3], new_state[i]
successors.append(tuple(new_state))
return successors
initial_state = (2, 8, 3, 1, 6, 4, 7, 0, 5)
if solve(initial_state):
print("The puzzle is solvable!")
else:
print("The puzzle is unsolvable.")
```

Output:IndentationError: expected an indented block after function definition.

Result: Hence, the simple python program for 8- puzzle is done.